LoanLedger PH — Product Requirements Document (Finalized)

Document type: Singular source of truth for full application re-creation

Product: LoanLedger PH — Loan Amortization, Disclosure & Portfolio Management

Status: Current build (synthesized from project inception through latest deployment)

Platform: Responsive web application (desktop & mobile browsers)

Last synthesized: 2026-06-18

> **How to read this document:** *This PRD captures only the *latest, finalized* state of every feature. Where an early behavior was later changed (e.g., the Date Paid auto-population bug, the deduct-from-proceeds default, DST auto-application), only the final behavior is documented. A* **Clarifications Needed** *section at the end flags known functional gaps.*

---

1. Executive Summary & App Vision

1.1 Purpose

LoanLedger PH is an invite-only, admin-operated lending operations platform for a Philippine micro-lending / arbitrage lending business. It lets a lender (the **Admin**) originate loans, produce BIR-aligned **Loan Disclosure Statements** and **amortization schedules**, track every installment's payment status, accept and verify borrower proofs of payment, and maintain private financial records of the lender's own economics (interest spread and the lender's own bank borrowings). **Borrowers** receive a transparent, read-mostly self-service portal showing their schedules, balances, and payment history.

1.2 Value Proposition

- **Transparency for borrowers:** every borrower sees a precise, read-only amortization schedule, outstanding balance, next payment due, and the disposition of every payment.
- **Operational control for the admin:** a single ledger is the system of record for all installment statuses; the borrower views read from the same store, so a status change is reflected instantly.
- **Financial intelligence for the admin (private):** dedicated, admin-only modules compute the lender's interest **spread/net gain** (Arbitrage) and track the lender's **own bank loans** (Loan Tracker), neither visible to borrowers.
- **Compliance-aware math:** Documentary Stamp Tax (DST), processing/notarial fees, flat add-on interest, and Philippine peso formatting are first-class.

1.3 Core Concept

A loan is captured in the **Loan Calculator**, which generates a disclosure statement and a schedule. "Assigning" a loan pushes one **transaction (installment) ledger row per scheduled payment** into the

shared store. The admin manages statuses in the **Overall Transactions** ledger; borrowers view the same data scoped to themselves. Money received is acknowledged in **Payment Logs**; borrower-submitted evidence flows through the **Verification Queue**.

---

2. User Personas & Roles

2.1 Roles & Permissions

Capability	Admin	Borrower (User)
Sign in (invite-only)	Yes	Yes
Self-registration	No (disabled)	No (disabled)
View **all** loans/transactions/payments	Yes	No (own only)
View own loans/transactions/payments	Yes	Yes
Originate loans (Calculator) & assign	Yes	No
Edit ledger (status, dates, amount, description)	Yes	No
Archive / restore / permanently purge ledger rows	Yes	No
CSV import of transactions/loans	Yes	No
Submit proof of payment	Yes (for a borrower, via View-As)	Yes (own)
Approve / reject proofs (Verification Queue)	Yes	No
Delete own proof of payment	Yes (any)	Yes (own)
Manage users (invite/edit/disable/delete)	Yes	No
Record **Payment Logs** (acknowledgements)	Yes	No (read-only own)
Interest / Arbitrage tracker	Yes	No (no visibility)
Loan Tracker (lender's own bank loans)	Yes	No (no visibility)
Manage stored interest-rate lists	Yes	No
Reports & Logs (analytics, audit, archives)	Yes	No
Edit own profile (name, nickname, phone, avatar)	Yes	Yes
Change own login email	Yes	No (admin-only)

2.2 Personas

- **Lender / Operator (Admin):** runs the lending business; originates loans, reconciles payments, and reviews private profitability. Trusted; sees all data.
- **Borrower (User):** an individual with one or more assigned loans/straight purchases; uses the portal to track obligations and upload proof of payment.
- **Admin "View as Borrower" (mode, not a role):** the Admin temporarily renders the borrower-scoped UI for a specific borrower (e.g., for support or to submit a proof on their behalf). The underlying session remains Admin; proofs submitted in this mode are attributed to the borrower.

---

3. Complete Feature Matrix

3.1 Authentication & Identity

- Invite-only email/password authentication (Supabase Auth).
- Pre-created profiles "adopted" by an auth account on first sign-in (by email match).
- First account ever created is provisioned as Admin; all others as Borrower.
- First-login **password setup** + account activation flow.
- Self-service profile editing (structured first/last/nickname, phone, avatar photo).
- Admin-only login-email change.

3.2 Admin Modules

1. **Overview / Dashboard** — portfolio KPIs.

2. **Overall Transactions** ledger — system of record for all installments; filter/search/sort, inline edit, single & bulk status changes, archive/restore/purge, CSV import + template, hide-settled toggle (default ON).

3. **Loan Calculator** — disclosure statement + amortization schedule; installment & straight types; negative (overpayment) straight amounts; editable rate dropdown; auto-DST (≥₱500,000); assign-and-push-live to a borrower; CSV export; share note.

4. **Verification Queue** — review borrower proofs (approve/reject with reason); pending badge.

5. **User Management** — invite, edit, disable, delete; "View as Borrower."

6. **Payment Logs** — record payments received; auto-computed Amount Owed; allocation (Settled/Overpayment/Underpayment); auto-netting carry-forward; per-borrower filter; read-only mirror for borrowers.

7. **Interest / Arbitrage** (admin-private) — lending-spread tracker with three+ summary metrics, per-loan & per-borrower views, and managed borrower/cost rate lists.

8. **Loan Tracker / "Loan Portfolio Dashboard"** (admin-private) — the lender's own bank loans; three summary tiles with per-bank breakdowns and bank logos; Outstanding/Fully Paid grids with collapsible cards; outstanding monthly-payment total tile.

9. **Reports & Logs** — analytics, append audit trail, archives (restore/purge), audit purge.

3.3 Borrower Modules

1. **My Dashboard** — KPIs + "My Loan Schedules" (hide-settled toggle, default ON).

2. **Straight Transactions** — one-time purchases (single payment); filters + hide-settled toggle.

3. **Consolidated Transactions** — all installments + straight items in one grid; filters + hide-settled toggle.

4. **My Payments** — submit proof of payment (file upload), view & delete own proofs; loan dropdown limited to assigned, not-fully-settled loans.

5. **Payment Logs** — read-only acknowledgements recorded by the Admin.

6. **Loan Detail** — per-loan disclosure statement + schedule.

7. **Profile** — edit name/nickname/phone/avatar.

3.4 Cross-cutting

- Shared calculation engine (DST, fees, flat add-on interest, end-of-month date clamping).
- Effective-status derivation (auto "Past Due").

- Persisted page state (filters/sorts survive navigation; reset on Refresh).
- Visible "Sync issue" badge on background write failures.
- Dual-mode data layer (live Supabase / in-memory demo).
- Light, frosted-glass administrative UI; responsive; pinned light color-scheme.

---

4. Detailed Epics & User Stories

> *Format:* **As a** *<role>,* **I want** *<capability>,* **so that** *<benefit>. Acceptance Criteria (AC) are testable and reflect the finalized behavior.*

EPIC A — Authentication & Account Lifecycle

A1 — Invite & adopt profile

As an Admin, I want to invite a borrower by name/email/phone so that a profile exists immediately and loans can be assigned before the borrower signs in.

- AC1: Creating an invite inserts a `profiles` row with `role='user'`, `status='invited'`.
- AC2: The actual sign-in invitation email is sent out-of-band (Supabase dashboard / Edge Function); the app does not send it.
- AC3: When an auth account is later created for the same email, it **adopts** the existing profile (the profile's `id` is relinked to the auth user id; loans/transactions/payments follow by cascade).
- AC4: If no profile matches the email, a fresh profile is created; the **first** profile ever created becomes Admin, all others Borrower.

A2 — First-login password setup & activation

As an invited user, I want to set my password on first login so that my account becomes active.

- AC1: While `status='invited'` (and `needsPasswordSetup` is true), the user is redirected to `/set-password` and cannot reach role pages.
- AC2: Setting a password updates the auth credential and calls `activate_my_account`, flipping `status` to `active`.
- AC3: After activation the user lands on their role home (`/admin` or `/portal`).

A3 — Role-based routing

As any user, I want to be routed to my role's area only.

- AC1: Unauthenticated → `/login`. Authenticated borrower hitting an `/admin/*` route → redirected to `/portal`; admin hitting `/portal/*` → redirected to `/admin`.
- AC2: Route guards are UX only; the database (RLS) is the real authority for data access.

EPIC B — Loan Origination (Calculator)

B1 — Generate a disclosure & schedule

As an Admin, I want to compute a loan so that I can show a compliant disclosure and amortization.

- AC1: Inputs: transaction type (installment|straight), principal, monthly add-on rate (%), duration (months), transaction date, first payment date, DST, processing fee, notarial fee, deduct-from-proceeds.
- AC2: **DST** auto-fills as `ceil(principal/200) × ■1.50` **only when principal ≥ ₱500,000**; below that it auto-fills 0. It remains manually editable; a manual edit holds for that principal value.

- AC3: **Processing fee** defaults to ₱1,500 (toggle-gated; unchecked = ₱0).
- AC4: **Deduct from Loan Proceeds** defaults to **OFF**; when off, fees are collected with the first payment rather than netted from proceeds.
- AC5: Schedule math: monthly principal = `P/D` (final month absorbs rounding remainder so principal sums exactly to P); monthly interest = `P × R` (flat add-on); total monthly = principal + interest (+ upfront fees on row 1 when not deducted from proceeds).
- AC6: **Straight** type forces duration = 1; a **negative** amount is permitted for straight only (an overpayment credit reducing total due); zero/NaN is rejected; negative installments are rejected.
- AC7: End-of-month dates clamp (e.g., Jan 31 → Feb 28/29) against the original anchor day.
- AC8: The rate field offers stored borrower rates as a dropdown (datalist) but still accepts free input.

B2 — Assign & push live

As an Admin, I want to assign a computed loan to a borrower so that it appears on their dashboard.

- AC1: Assigning persists the loan and exactly one transaction per schedule row (the displayed schedule is persisted verbatim — never recomputed on assignment).
- AC2: An Undo removes the loan and all its generated ledger rows.
- AC3: The action is recorded in the audit log.

EPIC C — Ledger Management (Overall Transactions)

C1 — Manage installment statuses

As an Admin, I want to change a transaction's status so that the borrower's view updates immediately.

- AC1: Statuses: `paid`, `unpaid`, `refunded`, `cancelled`, `past_due` (derived).
- AC2: **Date Paid invariant:** `paid` requires a date (auto-stamps today if absent); `refunded` keeps any existing date; `unpaid`/`cancelled`/`past_due` **must be blank** (never auto-populated).
- AC3: Setting a Date Paid on an unpaid/past-due row marks it paid; clearing it on a paid row reverts to unpaid.
- AC4: Single and bulk status updates are supported.
- AC5: **Effective status:** a stored `unpaid` row whose due date is before today displays as `past_due` (derived at read time, not persisted).

C2 — Archive / restore / purge

As an Admin, I want to soft-delete ledger rows so that mistakes are reversible, and permanently purge when sure.

- AC1: Archive moves rows to Archives (borrowers never see archived rows).
- AC2: Restore returns them to the active ledger.
- AC3: Purge permanently deletes archived rows.

C3 — CSV import

As an Admin, I want to import transactions/loans via CSV.

- AC1: A template with expected headers is downloadable.
- AC2: Transaction-only imports create standalone ledger rows (no parent loan).
- AC3: Loan-level imports create a real loan + its installments (amounts taken verbatim, never recalculated) so they appear with a full disclosure.

C4 — Hide settled toggle

As an Admin, I want to hide paid/refunded/cancelled rows by default.

- AC1: A modern switch defaults to ON (hiding settled); label toggles between "Show all transactions" and "Hide paid/refunded/cancelled."

EPIC D — Payments & Verification

D1 — Submit proof of payment (Borrower)

- AC1: Borrower selects a loan (dropdown limited to **own** loans that are **not** fully paid/refunded/cancelled), enters amount (> 0), method (GCash/Maya/Bank Transfer/Cash Deposit), reference, and uploads a file (image or PDF, ≤ 10 MB).
- AC2: The file is stored in a private bucket under the borrower's user-id folder; the row is created with `status='pending'`.
- AC3: An orphaned upload is rolled back if the row insert fails.

D2 — Verify proof (Admin)

- AC1: The Verification Queue lists pending proofs with a badge count.
- AC2: Admin approves or rejects; **rejection requires a note** (shown to the borrower).
- AC3: Borrowers and admins may delete a proof (row + file). *(See Clarifications: borrower delete is unrestricted by status.)*

EPIC E — Payment Logs (Acknowledgements)

E1 — Record a payment received (Admin)

As an Admin, I want to log money received and reconcile it against what the borrower owes.

- AC1: Fields: Transaction Date (default today), Transaction Reference #, Subject (auto "Payment Acknowledgement for <Due Date>", editable), Amount Owed, Payment Method (GCash/Maya/Bank Transfer/Cash), Funds Applied.
- AC2: **Amount Owed** auto-fills (editable) as the sum of the borrower's unpaid + past-due installment amounts with a due date on/before the chosen Due Date, **less the net prior carry**.
- AC3: **Allocation:** `Funds = Owed → Settled (0.00)`; `Funds > Owed → Overpayment (excess)`; `Funds < Owed → Underpayment (shortfall)`.
- AC4: A separate **carry** entry is written for any non-zero remainder; on the next log the borrower's unconsumed carries are **auto-netted** into Amount Owed and marked consumed.
- AC5: Recording a log **never** writes the transactions ledger (independent table).
- AC6: Borrowers see their own logs **read-only**.

EPIC F — Interest / Arbitrage (Admin-private)

F1 — Track lending spread

As an Admin, I want to see how much a borrower pays in interest vs. my cost so that I know my net gain.

- AC1: Per record: borrower, principal, transaction date, duration, first payment date (auto last payment date = first + duration), borrower rate %, cost rate %, DST (auto ≥₱500k), processing fee, notarial fee.

- AC2: `borrowerInterest = principal × borrowerRate% × months`; `interestCost = principal × costRate% × months`; `fees = DST + processing + notarial`; `netGain = borrowerInterest − interestCost + fees`.
- AC3: Summary tiles aggregate totals; per-loan and per-borrower views are available.
- AC4: Borrower/cost rate fields are editable dropdowns fed by **two managed rate lists**; rates can be added/removed.
- AC5: Entirely admin-only (RLS); borrowers receive no rows.

EPIC G — Loan Tracker / Loan Portfolio Dashboard (Admin-private)

G1 — Track the lender's own bank loans

- AC1: Header "Loan Portfolio Dashboard" with the current date.
- AC2: Three summary tiles — **Total Principal Availed**, **Total Interest**, **Total Repayment** — each with a per-bank breakdown line list showing a bank logo and the bank's contribution.
- AC3: Bank logos fetch from `https://logo.clearbit.com/{domain}`; on error, a colored box with the bank's initials is shown.
- AC4: Add-loan form: Bank (dropdown of PH universal/commercial banks; each option embeds `name|acronym|color|domain`), Principal, Processing Fee, Monthly Add-on Rate (%), Duration (months), Transaction Date, First Payment Date.
- AC5: `interest = principal × rate% × months`; `repayment = principal + interest`; `monthly = repayment / months`; `lastPayment = firstPayment + (duration − 1) months`.
- AC6: **Status:** if `lastPayment < today` → **Fully Paid**, else **Outstanding**.
- AC7: Two grids ("Outstanding Loans", "Fully Paid Loans") with count badges, sorted by transaction date **descending**.
- AC8: A tile below the form sums the **Monthly Payment of all Outstanding loans**.
- AC9: Each card is **collapsible** (collapsed shows only bank + Principal); a **Collapse/Expand all** control toggles every card.
- AC10: Sample loans are seeded so the dashboard is non-empty on first load.

EPIC H — Reports, Logs & Profile

H1 — Reports & Logs: analytics derived from the shared ledger; append-only audit trail (admin read; admin may purge); archives restore/purge.

H2 — Profile: edit first/last/nickname, phone, avatar; admin may change login email.

---

5. User Experience (UX) & Workflows

5.1 Onboarding (Invite → Active)

1. Admin → User Management → Invite (name, email, phone) → profile created (`invited`).
2. Out-of-band: sign-in invitation email sent (Supabase dashboard / Edge Function).
3. Borrower opens the link → signs in → app detects `invited` → redirects to **Set Password**.
4. Borrower sets password → `activate_my_account` → status `active` → lands on **My Dashboard**.

5.2 Loan Origination → Borrower Visibility

1. Admin → Loan Calculator → enter inputs → live disclosure + schedule render.

2. Admin selects a borrower → **Assign & push live** → loan + ledger rows persist.

3. Borrower immediately sees the loan under **My Loan Schedules** / **Consolidated Transactions** and the next payment due on **My Dashboard**.

5.3 Payment → Verification → Status

1. Borrower → My Payments → select loan → enter amount/method/reference → upload file → submit (`pending`).

2. Admin → Verification Queue → opens proof → **Approve** or **Reject (with reason)**.

3. Admin → Overall Transactions → marks the matching installment(s) `paid` (Date Paid auto-stamps).

4. Borrower's dashboard outstanding balance and statuses update on next load/refresh.

5.4 Reconciling an Overpayment/Underpayment (Payment Logs)

1. Admin → Payment Logs → Record Payment → choose borrower + due date → Amount Owed auto-computes.

2. Enter Funds Applied → live Remaining Balance + Allocation status preview.

3. Save → an acknowledgement row is written; if non-zero remainder, a carry row is written.

4. Next time, the carry auto-nets into Amount Owed and is marked consumed.

5.5 Tracking the Lender's Own Bank Loan

1. Admin → Loan Tracker → Add New Loan → pick bank, enter principal/fees/rate/duration/dates.

2. Save → card lands in Outstanding or Fully Paid based on computed last payment date vs. today.

3. Summary tiles and the outstanding monthly-total tile update; cards can be collapsed individually or all at once.

5.6 Global UX rules

- Filters, sorts, and calculator inputs persist across in-app navigation and reset on **Refresh**.
- Background write failures surface a non-blocking "Sync issue" badge with the error.
- Amounts render as Philippine peso (`₱1,234.56`); dates as `Mon DD, YYYY`.
- Responsive: multi-column layouts collapse to single column on small screens.

---

6. Technical & System Requirements

6.1 Architecture

- **Frontend:** React 19 + Vite, React Router, Tailwind CSS v4 (frosted-glass admin aesthetic), inline SVG icon set. Single-page application.
- **Backend-as-a-service:** Supabase — PostgreSQL, Auth (email/password), Storage, and Row-Level Security (RLS). The browser talks **directly** to Supabase using the **public anon key**; therefore **all authorization is enforced in the database via RLS** (client route guards are cosmetic).
- **Deployment:** Vercel (static SPA build; content-hashed bundles).
- **Data layer:** a single React context (`AppProvider`) exposes all entities and mutations and runs in **dual mode** — *live* (Supabase) or *demo* (in-memory mock, prototype only; no production

sign-in path).

6.2 Data Model (PostgreSQL)

Enums: `user_role(admin,user)`, `user_status(invited,active,disabled)`, `loan_txn_type(installment,straight)`, `loan_status(active,closed)`, `txn_status(paid,unpaid,refunded,cancelled,past_due)`, `payment_method(GCash,Maya,Bank Transfer,Cash Deposit)`, `payment_status(pending,approved,rejected)`, `pay_log_method(GCash,Maya,Bank Transfer,Cash)`, `pay_log_alloc(Settled,Overpayment,Underpayment)`, `pay_log_kind(payment,carry)`, `rate_kind(borrower,cost)`, plus an extensible `audit_action` enum.

Tables (key columns):

- `profiles` — `id (uuid = auth.users.id)`, `name`, `first_name`, `last_name`, `nickname`, `email (unique)`, `phone`, `role`, `status`, `invited_at`, `last_login`, `avatar_path`, `created_at`.
- `loans` — `id (text 'ln-####')`, `user_id→profiles`, `label`, `txn_type`, `principal (≠0)`, `monthly_rate`, `duration_months (1–60)`, `txn_date`, `first_payment_date`, `dst`, `processing_fee`, `notarial_fee`, `deduct_from_proceeds`, `status`, `created_at`; constraint: straight => duration = 1.
- `transactions` — `id (text '{loanId}-{n}')`, `loan_id (nullable→loans)`, `user_id→profiles`, `n`, `description`, `amount (≠0)`, `type (Installment|Straight)`, `txn_date`, `due_date`, `status`, `date_paid`, `archived_at`; date_paid/status coupling constraint; id-setting trigger; view `transactions_effective` derives `past_due` at read time (`security_invoker`).
- `payments` — `id (uuid)`, `user_id→profiles`, `loan_id→loans`, `amount (>0)`, `method`, `reference`, `file_name`, `file_type (image|pdf)`, `file_path`, `submitted_at`, `status`, `reviewed_at`, `note`; constraint: rejection requires a note.
- `audit_log` — `id`, `at`, `actor (text)`, `action`, `detail`.
- `payment_logs` — `id`, `user_id`, `kind`, `txn_date`, `reference`, `subject`, `due_date`, `amount_owed`, `method`, `funds_applied`, `remaining_balance`, `alloc_status`, `carry_applied`, `parent_id`, `consumed`, `consumed_by`, `note`, `created_at`.
- `arbitrage_loans` — `id`, `user_id`, `principal`, `txn_date`, `first_payment_date`, `duration_months (1–600)`, `last_payment_date`, `borrower_rate`, `cost_rate`, `dst`, `processing_fee`, `notarial_fee`, `created_at`.
- `interest_rates` — `id`, `kind (borrower|cost)`, `rate (numeric)`; unique (kind, rate).
- `tracked_loans` — `id`, `bank_name`, `bank_acronym`, `bank_color`, `bank_domain`, `principal`, `processing_fee`, `monthly_rate`, `duration_months (1–600)`, `txn_date`, `first_payment_date`, `created_at`.

Storage buckets: `payment-proofs` (private; 10 MB; image/jpeg,png,webp + pdf) and `avatars` (public; 2 MB; images). Per-user folders keyed by `auth.uid()`.

6.3 Authorization (RLS) Summary

- `is_admin()` — `SECURITY DEFINER`; true when the caller's profile `role='admin'` and `status ≠ 'disabled'`.

- `profiles`: read own or admin; insert/update/delete admin-only; borrowers edit own fields **only** via scoped `SECURITY DEFINER` RPCs (`update_my_profile`, `set_my_avatar`) — no broad UPDATE policy (prevents self role/email change).
- `loans`, `transactions`: read own (non-archived) or admin; all writes admin-only.
- `payments`: read own or admin; borrower may insert own `pending` rows; admin updates; borrower & admin may delete.
- `audit_log`: admin read; any authenticated may append; admin may delete.
- `payment_logs`: read own or admin; **admin-only writes**.
- `arbitrage_loans`, `interest_rates`, `tracked_loans`: **admin-only for all operations** (borrowers receive no rows).

6.4 Calculation Engine (deterministic, UI-independent)

- `computeDST(P) = ceil(P/200) × 1.50`; `autoDST(P) = P ≥ 500000 ? computeDST(P) : 0`.
- `computeDeductions` → `dst + processing + notarial`; `netProceeds = deductFromProceeds ? P − deductions : P`.
- `generateSchedule` → flat add-on interest, per-row principal `P/D` with final-row remainder absorption, optional first-row upfront fees; rejects zero/NaN; permits negative principal only when `D = 1`.
- `addMonthsClamped` → end-of-month-safe date advancement.
- Status helpers: `effectiveStatus` (auto past-due), `borrowerStatus` (adds `due`/`upcoming`), `isReceivable` (unpaid|past_due).
- Loan Tracker math: `lastPayment = first + (duration − 1)`; status by `lastPayment < today`.
- Arbitrage math: `lastPayment = first + duration`; `netGain = borrowerInterest − interestCost + fees`.

6.5 Third-party / Infrastructure Dependencies

- **Supabase** (Postgres, Auth, Storage, RLS) — required.
- **Clearbit Logo API** (`logo.clearbit.com/{domain}`) — Loan Tracker bank logos; client-side, fixed bank domains; graceful initials fallback on failure (no user data transmitted).
- **Vercel** — hosting/CI for the static build.
- Migrations are **manually applied** in the Supabase SQL Editor (numbered `001`–`013`).

6.6 Data Handling

- Read pagination bypasses Supabase's ~1,000-row API cap (`fetchAllRows`, page size 1,000).
- Mutations are optimistic; failures revert/notify via a shared error sink ("Sync issue" badge).
- Disclosure/aggregate figures are **derived**, not stored, to avoid drift; persisted schedules are stored verbatim at assignment.
- Money stored as `numeric(12,2)`; rates as `numeric(6,4)`.

---

7. Non-Functional Requirements

7.1 Security

- **AuthZ in the database:** RLS is the single source of authorization truth; the public anon key is shippable; the `service_role` key must never reach the client.
- **Invite-only:** public sign-up must be disabled in Supabase Auth (operational control); profiles are admin-provisioned.
- **Storage isolation:** payment proofs are in a private bucket with per-user folders and read-own-or-admin policies; signed URLs expire (1 hour).
- **Least privilege for self-service:** borrowers mutate only their own profile via narrowly scoped RPCs; admin-only modules (Arbitrage, Loan Tracker, Interest Rates, Payment Logs writes) are fully RLS-gated.
- **Transport:** HTTPS end-to-end (Supabase + Vercel).
- **Known security considerations (see Clarifications):** audit-log actor is client-supplied; borrower proof deletion is unrestricted by status; sign-up disablement is a dashboard setting; no MFA.

7.2 Performance

- SPA with content-hashed bundles; lazy data hydration per signed-in scope.
- Paginated reads; optimistic UI for perceived latency near zero on writes.
- Pure, memoized calculation helpers; table filtering/sorting computed client-side over already-loaded scoped data.

7.3 Scalability

- Stateless frontend on CDN (Vercel); Supabase scales the data tier.
- Indices on hot columns (`user_id`, `loan_id`, `due_date`, `status`, `created_at`).
- Page-through reads keep large ledgers correct beyond the API row cap.

7.4 Reliability & Data Integrity

- DB CHECK constraints enforce invariants (amounts, durations, date_paid/status coupling, rejection notes) as defense-in-depth behind client validation.
- Soft-delete (archive) with restore precedes any permanent purge.
- Cascading FKs keep loans/transactions/payments consistent with their profile.

7.5 Usability & Accessibility

- Responsive down to mobile; pinned **light** color-scheme so native control popups (date pickers, rate dropdowns) remain readable.
- Keyboard-focusable controls; ARIA labels on icon buttons, switches, and dialogs.
- Consistent peso/date formatting; empty states explain *why* a list is empty (e.g., all settled).

7.6 Compliance (Philippines fintech context)

- BIR Documentary Stamp Tax computed per ₱1.50/₱200 rule with the ≥₱500,000 auto-application threshold.
- Disclosure statement presents principal, fees (DST/processing/notarial), net proceeds, interest, and the full amortization schedule for borrower transparency.
- Audit trail records material actions (advisory — see Clarifications regarding tamper-evidence).

Clarifications Needed (Flagged Gaps)

These are logical/functional gaps in the synthesized history that should be resolved before treating this PRD as 100% complete:

1. **Password reset / "forgot password"** — No in-app flow exists. Confirm reliance on Supabase's built-in recovery email, or specify an in-app screen.
2. **Invitation email delivery** — Invites create a profile only; the sign-in email is sent manually (dashboard / Edge Function). Confirm whether an automated send (Edge Function) is in scope.
3. **MFA** — Not implemented. Confirm whether admin (or all) accounts require TOTP/MFA.
4. **Audit-log integrity** — `actor` is client-supplied and any authenticated user may append; admins may delete entries. Confirm whether the trail must be tamper-evident (server-set actor, append-only).
5. **Borrower proof deletion** — Borrowers can hard-delete their own proofs regardless of status (including approved). Confirm whether deletion should be limited to `pending`/soft-delete.
6. **Public sign-up enforcement** — Invite-only depends on a Supabase dashboard toggle, not code. Confirm it is disabled and whether code should additionally guard it.
7. **Enum & limit inconsistencies** — Payment method differs across modules (`Cash Deposit` in `payments` vs `Cash` in `payment_logs`); duration caps differ (`loans` 1–60 vs arbitrage/tracker 1–600). Confirm intended canonical values.
8. **Notifications / reminders** — No email/SMS reminders for upcoming or past-due payments. Confirm if a reminder system is required.
9. **Payment ↔ ledger linkage** — A verified proof does not automatically mark the matching installment `paid`; the admin does so manually. Confirm whether auto-reconciliation is desired.
10. **Loan Tracker / Arbitrage external linkage** — These admin-private records are standalone (not tied to borrower loans). Confirm this is intended and that seeded sample data is acceptable in production.
11. **Automated testing / CI** — No test suite or CI pipeline is described. Confirm quality gates.
12. **Demo mode** — An in-memory demo path exists in the data layer but is unreachable from the production login. Confirm it should remain disabled in production.